

**«Санкт-Петербургский государственный электротехнический университет  
«ЛЭТИ» им. В.И.Ульянова (Ленина)»  
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ  
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: Генетические алгоритмы**

Студент гр. 4343

\_\_\_\_\_

Г.Д. Маркашов

Студент гр. 4343

\_\_\_\_\_

Д.Е. Селезнев

Студент гр. 4343

\_\_\_\_\_

Р.А. Пимахин

Руководитель

\_\_\_\_\_

Т.Р. Жангиров

Санкт-Петербург

2026

**ЗАДАНИЕ  
НА УЧЕБНУЮ ПРАКТИКУ**

Студент: Г.Д. Маркашов

Студент: Д.Е. Селезнев

Студент: Р.А. Пимахин

Группа: 4343

Тема практики: Генетические алгоритмы

Задание на практику:

Дан взвешенный неориентированный граф (ребра имеют вес больше 0).  
Необходимо найти минимальное остовное дерево для данного графа. С  
использованием генетических алгоритмов

Ответственный за работу с данными, целостность доставки до пользователя  
и алгоритмов - Г.Д. Маркашов

Ответственный за работу алгоритма - Д.Е. Селезнев

Ответственный за работу графической части ПО - Р.А. Пимахин

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студент

\_\_\_\_\_

Г.Д. Маркашов

Студент

\_\_\_\_\_

Д.Е. Селезнев

Студент

---

---

Р.А. Пимахин

Руководитель

---

Т.Р. Жангиров

## АННОТАЦИЯ

В данной работе рассматривается задача поиска минимального остовного дерева (МОД) во взвешенном неориентированном графе с использованием генетического алгоритма. Целью проекта является разработка программного приложения с графическим интерфейсом пользователя, позволяющего задавать исходные данные, настраивать параметры алгоритма, выполнять поиск решения и визуализировать процесс оптимизации.

В отчёте описываются особенности постановки задачи, структура представления графа, принципы работы реализованного генетического алгоритма, используемые операции формирования популяции, отбора, скрещивания и мутации, а также функция качества решения. Рассматривается архитектура разработанного программного обеспечения, возможности взаимодействия пользователя с графическим интерфейсом и способы визуализации процесса поиска минимального остовного дерева.

Результатом работы является программное приложение на языке Python, позволяющее находить приближённое решение задачи МОД, отображать промежуточные этапы работы алгоритма, строить график изменения качества решений по поколениям и сравнивать различные варианты полученных решений.

## SUMMARY

In this paper, we consider the problem of finding a minimum spanning tree (MOD) in a weighted undirected graph using a genetic algorithm. The goal of the project is to develop a software application with a graphical user interface that allows you to set source data, configure algorithm parameters, search for solutions, and visualize the optimization process.

The report describes the specifics of the problem statement, the structure of the graph representation, the principles of operation of the implemented genetic algorithm, the operations of population formation, selection, crossing and mutation used, as well as the quality function of the solution. The architecture of the developed software, the possibilities of user interaction with a graphical interface, and ways to visualize the process of searching for a minimum spanning tree are considered.

The result of the work is a Python software application that allows you to find an approximate solution to the MOD problem, display the intermediate stages of the algorithm, graph changes in the quality of solutions over generations, and compare different variants of the solutions obtained.

## СОДЕРЖАНИЕ

|                                        |    |
|----------------------------------------|----|
| ВВЕДЕНИЕ .....                         | 7  |
| 1 ПОСТАНОВКА ЗАДАЧИ .....              | 8  |
| 1.1 Предметная область .....           | 8  |
| 1.2 Задачи практики .....              | 8  |
| 2 РЕЗУЛЬТАТЫ РАБОТЫ.....               | 10 |
| 2.1. Формулировка задачи 1 .....       | 10 |
| 2.2. Формулировка задачи 2 .....       | 15 |
| 2.3. Формулировка задачи 3 .....       | 15 |
| 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ .....        | 16 |
| 4 ОТЗЫВ РУКОВОДИТЕЛЯ .....             | 17 |
| ЗАКЛЮЧЕНИЕ .....                       | 18 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ..... | 19 |

## **ВВЕДЕНИЕ**

Предметной областью данной работы являются задачи оптимизации на графах, в частности задача поиска минимального остовного дерева (МОД) во взвешенном неориентированном графе. Подобные задачи находят применение при проектировании сетей связи, транспортных систем, электрических сетей и других областях, где требуется построение наиболее эффективной структуры при минимальных затратах. Для решения сложных оптимизационных задач могут применяться эвристические методы, одним из которых являются генетические алгоритмы, основанные на моделировании процессов естественного отбора и эволюции.

Целью практики является разработка программного приложения для поиска минимального остовного дерева во взвешенном неориентированном графе с использованием генетического алгоритма. Разрабатываемая программа должна обеспечивать возможность задания исходных данных различными способами, настройки параметров алгоритма пользователем, визуального отображения процесса поиска решения и анализа эффективности работы алгоритма.

Для достижения поставленной цели необходимо решить следующие задачи: изучить особенности задачи поиска минимального остовного дерева и методы её решения; разработать представление графа и структуру генетического алгоритма; реализовать основные этапы работы алгоритма, включая формирование начальной популяции, оценку качества решений, отбор, скрещивание и мутацию; разработать графический интерфейс пользователя на языке Python; реализовать возможность ввода данных через интерфейс, загрузки из файла и генерации случайных графов; обеспечить пошаговую визуализацию процесса поиска решения, построение графика изменения функции качества и возможность получения итогового результата без просмотра всех промежуточных этапов.

# 1 ПОСТАНОВКА ЗАДАЧИ

## 1.1 Предметная область

Предметной областью практики являются поиск минимального остовного дерева во взвешенном неориентированном графе с использованием генетических алгоритмов. Подобные задачи применяются при проектировании сетей связи, транспортных систем, электрических сетей и других структур, где требуется найти наиболее эффективное соединение объектов с минимальными затратами.

В рамках практики рассматривается применение генетических алгоритмов для решения задач. Разрабатываемое программное приложение позволяет исследовать процесс поиска решения, настраивать параметры алгоритма и визуализировать этапы его работы.

## 1.2 Задачи практики

В рамках прохождения практики были поставлены следующие задачи:

1. Изучить особенности задачи поиска минимального остовного дерева во взвешенном неориентированном графе и существующие методы её решения.
2. Изучить принципы работы генетических алгоритмов и возможности их применения для решения задач поиска оптимальных решений.
3. Изучить средства разработки графического интерфейса на языке Python, в частности библиотеку Tkinter, а также инструменты визуализации графов с использованием библиотеки NetworkX.
4. Разработать структуру представления графа и алгоритм поиска минимального остовного дерева на основе генетического алгоритма.
5. Реализовать основные этапы работы генетического алгоритма: создание начальной популяции, вычисление функции качества, отбор, скрещивание и мутацию.
6. Разработать графический интерфейс программы с использованием Tkinter, обеспечивающий ввод данных, загрузку графов из файла, генерацию случайных графов и настройку параметров алгоритма.



7. Реализовать визуализацию графа и процесса поиска решения с использованием NetworkX, включая отображение промежуточных результатов, текущих решений и изменения качества решений по поколениям.

8. Провести тестирование разработанного приложения и оценить корректность работы алгоритма на различных входных данных.

## 2 РЕЗУЛЬТАТЫ РАБОТЫ

### 2.1. Работа с данными. Проверка целостности и доставка данных от UI до алгоритма и обратно.

Была проведена работа над pydantic схемами для валидации и хранения данных. В ходе проведения разработки ПО были созданы следующие классы:

`class GAStateStep(BaseModel)` для хранения информации о текущем шаге. Он имеет следующие поля:

`generation` – номер поколения

`population` – список хромосом(кодов Прюфера)

`best_fitness` – лучшее значение качества в поколении

`avg_fitness` – среднее значение качества в поколении

`worst_fitness` – худшее значение качества в поколении

`class GAHistory(BaseModel)` для хранения истории работы алгоритма (история шагов эволюции) со следующими полями:

`generations_count` – кол-во поколений

`best_fitness_history` – список лучших значений качества в поколениях

`avg_fitness_history` – список средних значений качества в поколениях

`worst_fitness_history` – список худших значений качества в поколениях

`steps` – список шагов эволюции (`GAStateStep`)

Класс имеет несколько методов:

`add_step(self, step: GAStateStep)` для добавления шага в историю

`get_step(self, index: int) -> GAStateStep` для получения шага по индексу

`get_last_step()` для получения последнего шага

`class GraphData(BaseModel)` для хранения информации о графе. Имеет следующие поля:

`num_vercites` – кол-во вершин в графе

`matrix` – матрица смежности

Для корректности работы алгоритмов были установлено несколько валидаций данных и проверки на их соблюдения. Именно для этих целей был создан метод `validate_matrix` который срабатывает сразу после создание объекта и проверяет граф на:

1. Наличие петель и в случае их обнаружения выбрасывает исключение `ValueError`
2. Использование только ориентированного графа, в случае исключения выбрасывает исключение `ValueError`

Для удобства работы с алгоритмами и конвертации данных были реализованы следующие методы:

`from_edges_list(n: int, edges: list[tuple[int, int, float]])` для создания объекта класса с использованием списка ребер

`to_edges_list` обратная функция для извлечения списка ребер из графа.

Class `Graph` для управления данными графа и извлечению, установки и применения графа в алгоритме. Конструктор класса принимает кол-во вершин и создает объекта класса `GraphData` и хранит кол-во вершин. Для управления графом были реализованы следующие функции:

`get_edges` – метод возврата списка ребер из графа

`get_matrix` – метод возврата списка смежности графа

`get_data` – метод возврата данных графа из объекта `GraphData`

`_build_adj_list` – защищенный метод для формирования списка смежности

`_validate_vertex` – метод для валидации вершины по допустимому диапазону значений

`add_edge` – метод по добавлению ребра в граф

`find_edge` – метод по поиску ребра в графе

`has_edge` – метод по проверки существования ребра

`get_edge_weight` – метод по получению веса ребра

`get_copy_edges` – метод возвращаемый копия списка ребер  
`get_copy_neighbors` – метод по возвращению копии списка соседей(списка смежности)  
`get_degree` – метод возвращающий степень вершины  
`get_total_edges` – метод возвращающий кол-во ребер в графе  
`get_total_weight` – метод возвращающий вес всех ребер графа в сумме  
`get_edges_weight` – метод возвращающий вес определенных ребер, передаваемых списком кортежей ребер  
`to_dict` – метод миграции данных о графе в словарь  
`to_json` – метод миграции данных в json файл  
`is_connected` – метод проверки связности графа  
`is_tree` – статический метод класса для проверки списка ребер на образование дерева из этих ребер

`class PruferCode` один из важнейших классов в ПО. Класс призван для кодирования и декодирования графа в код Прюфера из ребер и наоборот из кода Прюфера в список ребер. Имеет следующие методы:

`edges_to_code` – статический метод класса для кодирования списка ребер в код Прюфера  
`code_to_edges` – статический метод для декодирования кода Прюфера в список ребер  
`is_valid_prufer_code` – статический метод для проверки влидности кода Прюфера

`class FileIO`. Класс предназначен для работы с файлами, а именно для чтения графа из файла в формате списка ребер. Класс имеет следующий метод:

`load` – статический метод считывающий данные из файла, их валидация и создания объекта класса `Graph` с использованием полученных ребер и последующим возврата объекта класса `Graph`

`class GraphGenerator`. Класс для генерации графа с использованием параметров. Класс имеет следующий метод:

`gen_graph` – статический метод графа с параметрами: кол-во вершин, вероятность появления ребра между вершинами, минимальным и максимальным весам ребра

`class GraphManager`. Класс абстракции объединяющий несколько классов в один для удобного управления и комбинации использований других классов, а также для валидации приходящих данных. Метод конструктор объекты классов `FileIO` и `GraphGenerator`, а также поле `_graph` для хранения графа. Класс представляет собой менеджер управления графом. Имеет следующие методы:

`get_graph` – возвращает объект метода `Graph` при его наличии

`has_graph` – проверяет есть ли граф в менеджере

`set_graph` – метод установки графа в менеджере

`clear` – метод по удалению графа из менеджера

`load_from_tuple` – метод по созданию и добавления графа в менеджер при входных данных списка ребер с весами

`generate_random_graph` – метод по генерации графа с использованием другого класса

`get_edges` – метод возврата списка ребер из графа

`get_matrix` – метод возврата списка смежности графа

`get_data` – метод возврата данных графа из объекта `GraphData`

`add_edge` – метод по добавлению ребра в граф

`find_edge` – метод по поиску ребра в графе

`has_edge` – метод по проверки существования ребра

`get_edge_weight` – метод по получению веса ребра

`get_copy_edges` – метод возвращаемый копия списка ребер

`get_copy_neighbors` – метод по возвращению копии списка соседей(списка смежности)

`get_degree` – метод возвращающий степень вершины  
`get_total_edges` – метод возвращающий кол-во ребер в графе  
`get_total_weight` – метод возвращающий вес всех ребер графа в сумме  
`get_edges_weight` – метод возвращающий вес определенных ребер, передаваемых списком кортежей ребер  
`graph_to_dict` – метод миграции данных о графе в словарь  
`graph_to_json` – метод миграции данных в json файл  
`graph_is_connected` – метод проверки связности графа

`class HistoryManager`. Класс по хранению и управлению историей эволюции особей, шагов алгоритма. В конструкторе класс создает объекта класса `GAHistory`. Класс имеет следующие методы:

`add_steps` – метод по добавлению шага историю  
`go_forward` – метод по переходу на следующий шаг в истории  
`go_back` – метод по переходу на шаг назад в истории  
`go_to` – метод по переходу на конкретный шаг в истории  
`go_to_end` – метод по переходу на последний шаг в истории  
`go_to_start` – метод по переходу в начало истории  
`get_current` – метод по извлечению данных из истории с текущего шага  
`get_step` – метод по получению шага по индексу  
`get_all` – метод получения копии всей истории  
`get_history_stats` – метод по получению статистики  
`get_last` – метод по получению информации из последнего шага  
`get_first` – метод по получению информации из первого шага  
`clear` – метод по очистки истории  
`is_empty` – метод по проверки пустоты истории

`class CrossoverType`. Класс Enum для избежания ошибок при конвертации параметров полученных из графического пользовательского интерфейса и переходе в работу к алгоритму

class MutationType. Класс Enum для избежания ошибок при конвертации параметров полученных из графического пользовательского интерфейса и переходе в работу к алгоритму

class SelectionType. Класс Enum для избежания ошибок при конвертации параметров полученных из графического пользовательского интерфейса и переходе в работу к алгоритму

.

## **2.2. Формулировка задачи 2**

...

## **2.3. Формулировка задачи 3**

...

### **3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ**

При выполнении практической работы использовались следующие технологии и программные средства:

Python 3 основной язык программирования, на котором реализованы генетический алгоритм и программное приложение.

Tkinter стандартная библиотека Python для создания графического интерфейса пользователя. Использовалась для разработки окон приложения, организации ввода исходных данных, настройки параметров генетического алгоритма и отображения результатов работы программы.

NetworkX библиотека для работы с графами. Применялась для создания, визуализации взвешенного неориентированного графа.

Rydantic библиотека для работы с данными, удобному обращению к ним и валидации их.



#### 4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша\_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

##### Отзыв

##### о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

## **ЗАКЛЮЧЕНИЕ**

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

## **СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.